
30 RAG Interview Questions & Answers

The Complete Guide to Crack RAG System Design Interviews in 2026

First-Principles Thinking | Easy Language | Real Examples

RAG Fundamentals	Chunking Strategies	Embeddings & Vector DBs
Retrieval Strategies	Advanced RAG Patterns	Reranking & Post-Retrieval
RAG Evaluation	Production RAG	Multimodal & Agentic RAG
	Troubleshooting & Optimization	

AmanAI Lab

amanailab.com | YouTube: [@AmanAILab](https://www.youtube.com/@AmanAILab)

FREE RESOURCE - Share with your network!

Table of Contents

■ RAG Fundamentals

- Q1. What is RAG (Retrieval-Augmented Generation) and why do we need it?
- Q2. Explain the complete RAG pipeline step by step.
- Q3. What are the key components of a RAG system and what choices do you make for each?

■ Chunking Strategies

- Q4. Why is chunking important and what happens if you get it wrong?
- Q5. What are the different chunking methods and when do you use each?
- Q6. What is chunk overlap and why is it critical?

■ Embeddings & Vector Databases

- Q7. How do you choose the right embedding model for your RAG system?
- Q8. Compare the major vector databases — when would you use each?
- Q9. What is the difference between HNSW, IVF, and flat search in vector databases?

■ Retrieval Strategies

- Q10. What is Hybrid Search and why is it better than pure vector search?
- Q11. What is the 'Lost in the Middle' problem and how do you solve it?
- Q12. What is Query Transformation and why does it improve RAG?

■ Advanced RAG Patterns

- Q13. What is the difference between Naive RAG, Advanced RAG, and Modular RAG?
- Q14. What is Parent-Child chunking (also called hierarchical chunking)?
- Q15. What is Multi-Index RAG and when do you use it?

■ Reranking & Post-Retrieval

- Q16. What is Reranking and why is it a game-changer for RAG quality?
- Q17. What is Contextual Compression and how does it help RAG?
- Q18. How do you handle metadata filtering in RAG retrieval?

■ RAG Evaluation

- Q19. How do you evaluate a RAG system? What metrics do you use?
- Q20. What is RAGAS and how does it work?
- Q21. How do you create a test set for RAG evaluation?

■ Production RAG

- Q22. What are the common failure modes of RAG systems in production?

Q23. How do you handle document updates and keep the RAG knowledge base fresh?

Q24. How do you add citations and source attribution to RAG answers?

■ Multimodal & Agentic RAG

Q25. What is Agentic RAG and how is it different from standard RAG?

Q26. How do you build RAG over tables, charts, and images (Multimodal RAG)?

Q27. What is Graph RAG and when would you use it over standard RAG?

■ RAG Troubleshooting & Optimization

Q28. Your RAG system is hallucinating despite having the right documents. How do you fix it?

Q29. How do you optimize RAG latency for real-time applications?

Q30. How do you handle multi-turn conversations in RAG?

RAG Fundamentals

Q1 What is RAG (Retrieval-Augmented Generation) and why do we need it?

Answer:

RAG is an architecture pattern where you **retrieve relevant documents from an external knowledge base** and inject them into the LLM's prompt before generating an answer. Instead of relying solely on what the LLM memorized during training, you give it fresh, relevant context at query time. Why we need it: (1) **LLMs have a knowledge cutoff** — they don't know anything after their training date. (2) **LLMs hallucinate** — without grounding data, they make things up confidently. (3) **Domain-specific knowledge** — your company's internal docs, policies, and data aren't in any public LLM. (4) **Cheaper than fine-tuning** — updating a knowledge base is instant; retraining a model takes days and thousands of dollars.

■ Example:

Without RAG:

User: 'What is our company refund policy?'

LLM: 'Generally, most companies offer 30-day refunds...' (generic, possibly wrong)

With RAG:

Step 1: Search company knowledge base for 'refund policy'

Step 2: Retrieve: 'Refund policy: Full refund within 14 days. Store credit after 14 days. No refund after 30 days.'

Step 3: Inject into prompt as context

Step 4: LLM generates: 'Our refund policy allows full refunds within 14 days, store credit between 14-30 days, and no refunds after 30 days.'

Grounded, accurate, company-specific answer.

■ *Interview Tip: Say 'RAG solves three problems: knowledge recency, hallucination grounding, and domain specificity — without the cost of fine-tuning.' This one sentence shows you understand both the WHY and the economics.*

Q2**Explain the complete RAG pipeline step by step.****Answer:**

The RAG pipeline has two phases — **Indexing** (offline, done once) and **Retrieval + Generation** (online, per query). **Indexing phase:** (1) Load documents (PDFs, web pages, databases). (2) Split documents into chunks (paragraphs, sentences, or fixed-size windows). (3) Generate embeddings for each chunk using an embedding model. (4) Store embeddings + original text in a vector database. **Query phase:** (1) User asks a question. (2) Generate embedding of the query using the same embedding model. (3) Search vector database for top-K most similar chunks (cosine similarity). (4) Inject retrieved chunks into the LLM prompt as context. (5) LLM generates answer grounded in the retrieved context. (6) Optionally, cite sources.

■ Example:

Indexing (one-time setup):

```
500 company PDFs -> split into 15,000 chunks
-> embed each chunk using text-embedding-3-small
-> store in Pinecone (vector DB)
Total time: ~30 minutes, ~$5 embedding cost
```

Query time (every user question):

```
User: 'What are the Q3 revenue numbers?'
-> Embed query -> search Pinecone -> top 5 chunks returned
-> Chunks contain Q3 financial report sections
-> LLM reads chunks + generates: 'Q3 revenue was $4.2M, up 18% YoY'
Total time: ~2 seconds, ~$0.003 per query
```

■ *Interview Tip: Draw the pipeline on a whiteboard in two rows — Indexing (offline) and Query (online). Interviewers love visual explanations, and it shows you understand the system architecture.*

Q3

What are the key components of a RAG system and what choices do you make for each?

Answer:

Six key components with choices: (1) **Document Loader** — how you ingest data. Choices: LangChain loaders, Unstructured.io, LlamaIndex. (2) **Chunking Strategy** — how you split documents. Choices: fixed-size, recursive, semantic, sentence-level. (3) **Embedding Model** — converts text to vectors. Choices: OpenAI text-embedding-3, Cohere embed-v3, open-source (BGE, E5, Jina). (4) **Vector Database** — stores and searches embeddings. Choices: Pinecone, Weaviate, ChromaDB, Qdrant, pgvector. (5) **Retrieval Strategy** — how you search. Choices: dense search, sparse (BM25), hybrid, reranking. (6) **LLM Generator** — generates the final answer. Choices: GPT-4, Claude, Llama, Mistral. Each choice impacts quality, cost, latency, and accuracy differently.

■ **Example:**

```
Budget-friendly production RAG stack:  
Loader: Unstructured.io (handles PDFs, DOCX, HTML)  
Chunking: Recursive text splitter (512 tokens, 50 token overlap)  
Embedding: text-embedding-3-small ($0.02/1M tokens)  
Vector DB: Qdrant (open-source, self-hosted)  
Retrieval: Hybrid (BM25 + dense) with Cohere reranker  
LLM: GPT-4o-mini for simple queries, GPT-4 for complex
```

```
Premium production RAG stack:  
Embedding: text-embedding-3-large  
Vector DB: Pinecone (managed, serverless)  
Retrieval: Hybrid + cross-encoder reranking  
LLM: Claude Opus with streaming
```

■ *Interview Tip: When asked to design a RAG system, list all 6 components and justify EACH choice. Don't just say 'I would use Pinecone' — say 'I would use Pinecone for managed serverless scaling, but for a cost-sensitive project, Qdrant self-hosted.'*

Chunking Strategies

Q4

Why is chunking important and what happens if you get it wrong?

Answer:

Chunking is the process of **splitting documents into smaller pieces** before embedding them. It's arguably the MOST important step in RAG because it directly determines what gets retrieved. If chunks are **too large**: (1) Each chunk contains mixed topics, diluting the embedding's meaning. (2) You waste context window tokens on irrelevant text. (3) Retrieval precision drops. If chunks are **too small**: (1) Context is fragmented — the answer spans multiple chunks but only one gets retrieved. (2) Loss of surrounding context makes chunks meaningless. (3) More chunks = higher storage and search costs. The sweet spot depends on your content type, but typically 256-1024 tokens with 10-20% overlap.

■ Example:

Document: A 5-page refund policy document.

Too large (entire document as 1 chunk):

User asks about 'return window' -> retrieves ALL 5 pages

LLM gets overwhelmed with irrelevant shipping, warranty info

Answer quality: Poor (lost in the noise)

Too small (every sentence as 1 chunk):

Chunk: 'Returns must be made within 14 days.'

Misses: '...from the date of delivery, with original packaging.'

Answer quality: Incomplete

Just right (paragraph-level, ~300 tokens):

Chunk: 'Returns must be made within 14 days from the date of delivery, with original packaging. Items must be unused and in resalable condition. Refund will be processed within 5-7 business days.'

Answer quality: Complete and accurate

■ *Interview Tip: Say 'Chunking is where most RAG systems fail. I always experiment with 3-4 chunk sizes and evaluate retrieval quality before finalizing.' This shows you don't just set defaults — you optimize.*

Q5**What are the different chunking methods and when do you use each?****Answer:**

Five main approaches: (1) **Fixed-size chunking** — split every N tokens/characters. Simple but can break sentences mid-thought. Use for: uniform documents like logs. (2) **Recursive text splitting** — split by paragraphs first, then sentences, then characters as fallback. Most popular default. Use for: general documents. (3) **Sentence-level splitting** — each chunk is one or more complete sentences. Preserves grammatical meaning. Use for: FAQs, Q&A pairs. (4) **Semantic chunking** — uses embeddings to find natural topic boundaries. Groups sentences with similar meaning together. Use for: long documents with multiple topics. (5) **Document-structure-aware** — uses headings, sections, tables as chunk boundaries. Respects document layout. Use for: structured docs like legal contracts, technical manuals, research papers.

■ Example:

Same document, different strategies:

Document: 'Machine Learning Overview' (10 pages)

Fixed (512 tokens): 20 chunks, some split mid-sentence

Recursive: 18 chunks, clean paragraph boundaries

Semantic: 12 chunks, each covering one concept

Structure-aware: 8 chunks (Intro, Types of ML, Supervised, Unsupervised, DL, Applications, Challenges, Conclusion)

Retrieval test with query 'What is supervised learning?':

Fixed: Retrieved chunk has half supervised + half unsupervised (confused)

Recursive: Retrieved chunk has full supervised section (good)

Semantic: Retrieved chunk has supervised + relevant examples (best)

Structure-aware: Retrieved exact 'Supervised Learning' section (best)

■ *Interview Tip: Mention 'semantic chunking' proactively. Most candidates only know fixed-size. Saying 'I use semantic chunking with embedding similarity to find natural topic boundaries' puts you ahead of 90% of candidates.*

Q6

What is chunk overlap and why is it critical?

Answer:

Chunk overlap means consecutive chunks share some tokens at their boundaries. If chunk 1 ends at position 500 and chunk 2 starts at position 450, there's a 50-token overlap. Why it matters: Information often spans chunk boundaries. Without overlap, a sentence split across two chunks loses its meaning in both. With overlap, the critical boundary content appears in BOTH chunks, so retrieval can find it regardless of which chunk it searches. Typical overlap: **10-20% of chunk size**. If chunk size is 512 tokens, use 50-100 token overlap. Too much overlap = redundancy and storage waste. Too little = information loss at boundaries.

■ Example:

Document text:

```
'...The model achieved 94% accuracy on the test set.  
However, the precision for class B was only 67%.  
This indicates significant class imbalance issues...'
```

Without overlap (chunk boundary after 'test set.')

Chunk 1: '...The model achieved 94% accuracy on the test set.'

Chunk 2: 'However, the precision for class B was only 67%. This indicates...'

Query: 'What was the model accuracy?'

-> Retrieves Chunk 1 but misses the precision problem

With 50-token overlap:

Chunk 1: '...The model achieved 94% accuracy on the test set. However, the precision for class B was only 67%.'

Chunk 2: 'However, the precision for class B was only 67%. This indicates significant class imbalance issues...'

Query: 'What was the model accuracy?'

-> Retrieves Chunk 1 with COMPLETE context including the precision caveat

■ *Interview Tip: Say 'I always use 10-20% overlap to prevent information loss at chunk boundaries.' Simple, practical, shows you've built real RAG systems.*

Embeddings & Vector Databases

Q7**How do you choose the right embedding model for your RAG system?****Answer:**

Consider five factors: (1) **Quality vs Cost** — OpenAI text-embedding-3-large is best quality but costs money. Open-source BGE-large or E5-mistral are free but need hosting. (2) **Dimension size** — higher dimensions (1536, 3072) capture more nuance but use more storage. Lower dims (384, 768) are faster and cheaper. (3) **Multilingual support** — if your docs are in multiple languages, use multilingual models like Cohere embed-v3 or multilingual-e5. (4) **Domain fit** — general embeddings work for most cases. For specialized domains (medical, legal), domain-fine-tuned embeddings perform better. (5) **MTEB benchmark scores** — check the MTEB leaderboard for retrieval task performance. Don't just pick the most popular model — pick the best one for YOUR data.

■ Example:

Choosing embedding model for different scenarios:

Startup FAQ bot (English only, low budget):

-> BGE-small-en (384 dims, free, good quality)

-> Storage: 384 floats x 10K chunks = ~15MB

Enterprise knowledge base (multilingual, high quality):

-> Cohere embed-v3 (1024 dims, multilingual)

-> Storage: 1024 floats x 500K chunks = ~2GB

Medical research assistant:

-> Fine-tuned PubMedBERT embeddings

-> Or OpenAI text-embedding-3-large + domain-specific eval

Key principle: ALWAYS evaluate on YOUR data.

Benchmark says Model A is best, but on your legal documents,

Model B might retrieve better because of domain vocabulary.

■ *Interview Tip: Mention the MTEB leaderboard by name. Say 'I check MTEB retrieval scores before selecting an embedding model, but I always validate on my own domain data.' Shows you follow a rigorous process.*

Q8**Compare the major vector databases — when would you use each?****Answer:**

Key options: (1) **Pinecone** — fully managed, serverless, zero-ops. Best for: teams who don't want to manage infrastructure. Paid only. (2) **Weaviate** — open-source, supports hybrid search natively, has built-in vectorization. Best for: full-featured self-hosted deployments. (3) **ChromaDB** — lightweight, open-source, embeds in your Python app. Best for: prototyping, small projects, local development. (4) **Qdrant** — open-source, high performance, Rust-based. Best for: production workloads needing speed and filtering. (5) **pgvector** — PostgreSQL extension. Best for: teams already using Postgres who don't want a separate database. (6) **Milvus** — open-source, handles billion-scale vectors. Best for: massive datasets. Choose based on: scale, budget, ops capability, and whether you need hybrid search.

■ Example:

Decision matrix:

Prototype/Hackathon -> ChromaDB (pip install, in-memory, zero config)
Startup (0-100K docs) -> Qdrant Cloud or Pinecone Serverless
Enterprise (100K-10M docs) -> Weaviate or Pinecone
Massive scale (10M+ docs) -> Milvus or Pinecone Enterprise
Already using PostgreSQL -> pgvector (no new infrastructure)

Cost comparison for 1M vectors (1536 dims):

ChromaDB: Free (self-hosted, ~4GB RAM)
Qdrant Cloud: ~\$25/month
Pinecone Serverless: ~\$30/month
Weaviate Cloud: ~\$50/month
pgvector: Free (existing Postgres)

■ *Interview Tip: Don't just name a vector DB. Say 'For this use case, I would pick X because...' and mention a specific reason (cost, managed vs self-hosted, hybrid search support). Decision-making impresses more than knowledge.*

Q9

What is the difference between HNSW, IVF, and flat search in vector databases?

Answer:

These are **indexing algorithms** that determine how the vector DB searches for similar vectors. (1) **Flat (brute-force)** — compares query against EVERY vector. 100% accurate but $O(n)$ — impossibly slow at scale. Use only for tiny datasets (<10K vectors). (2) **IVF (Inverted File Index)** — clusters vectors into buckets. At query time, only searches the closest buckets, not all vectors. Fast but might miss vectors in neighboring clusters. Tunable: more buckets searched = higher recall but slower. (3) **HNSW (Hierarchical Navigable Small World)** — builds a multi-layer graph of vectors. Searches by navigating the graph from coarse to fine layers. Best balance of speed and accuracy. Most popular in production. Trade-off: higher memory usage.

■ Example:

Searching 10 million vectors:

Flat search:

Compare query vs 10M vectors

Time: ~5 seconds

Recall: 100% (perfect, finds the best match)

Practical? No. Too slow for production.

IVF (1000 clusters, search top 10):

Step 1: Find 10 closest clusters

Step 2: Search ~100K vectors in those clusters

Time: ~50ms

Recall: ~95% (might miss if best match is in cluster #11)

HNSW (M=16, ef=200):

Navigate graph layers: top -> middle -> bottom

Time: ~10ms

Recall: ~98%

Memory: 2x more than IVF

HNSW wins for most production use cases. IVF wins when memory is tight.

■ *Interview Tip: Say 'I default to HNSW for production RAG because it gives the best speed-recall tradeoff, but I use IVF when memory is constrained.' This shows you understand the engineering tradeoffs.*

Retrieval Strategies

Q10**What is Hybrid Search and why is it better than pure vector search?****Answer:**

Hybrid search combines dense retrieval (vector/semantic search) with sparse retrieval (keyword search like BM25) to get the best of both worlds. Dense search finds semantically similar content even with different words ('car' matches 'automobile'). But it can miss exact terms — searching for 'error code E-4012' might not find the document with that exact code. Sparse search (BM25) matches exact keywords perfectly but misses semantic meaning. Hybrid combines both: run BM25 AND vector search in parallel, then merge results using **Reciprocal Rank Fusion (RRF)** or weighted scoring. This consistently outperforms either method alone by 10-20% in retrieval benchmarks.

■ Example:

Query: 'How to fix error E-4012 in the authentication module?'

Dense search only (vector):

Result 1: 'Authentication troubleshooting guide' (semantically relevant)

Result 2: 'Common login issues and fixes' (related topic)

Result 3: 'Security module overview' (vaguely related)

-> MISSED the exact document about error E-4012!

Sparse search only (BM25):

Result 1: 'Error E-4012: Token expiry in auth module' (exact match!)

Result 2: 'Error E-4013: Certificate mismatch' (keyword overlap)

-> Found the exact doc but missed semantically related content.

Hybrid (BM25 + Vector + RRF):

Result 1: 'Error E-4012: Token expiry in auth module' (exact + semantic)

Result 2: 'Authentication troubleshooting guide' (semantic context)

Result 3: 'Token refresh implementation guide' (related solution)

-> Best of both worlds!

■ *Interview Tip: Always recommend hybrid search for production RAG. Saying 'I would use only dense search' is a red flag — it shows you haven't dealt with real-world edge cases where exact terms matter.*

Q11**What is the 'Lost in the Middle' problem and how do you solve it?****Answer:**

Research shows that LLMs **pay more attention to information at the beginning and end of the context**, and often ignore or underweight information in the middle. If you retrieve 10 chunks and the correct answer is in chunk #5 (middle), the LLM might miss it. Solutions: (1) **Reduce number of retrieved chunks** — use fewer, more relevant chunks so nothing gets buried. (2) **Rerank before injection** — put the most relevant chunks FIRST in the prompt. (3) **Chunk summarization** — summarize retrieved chunks into a condensed context. (4) **Query decomposition** — split complex queries into sub-queries, retrieve separately, then combine. (5) **Smart positioning** — put critical context at the START of the prompt, instructions at the END.

■ Example:

```
10 retrieved chunks ordered by similarity:
Chunk 1: 0.95 relevance (start - LLM pays attention)
Chunk 2: 0.93
Chunk 3: 0.91
Chunk 4: 0.89
Chunk 5: 0.87 -> THE ACTUAL ANSWER IS HERE
Chunk 6: 0.85
...
Chunk 10: 0.75 (end - LLM pays attention)
```

Problem: LLM focuses on chunks 1-3 and 9-10, skips chunk 5.

Solution 1: Only retrieve top 3-5 chunks instead of 10.

Solution 2: Rerank so the answer chunk moves to position 1.

Solution 3: Summarize all 10 chunks into 1 condensed paragraph.

Best practice: Retrieve 20, rerank to top 5, put most relevant first.

■ *Interview Tip: Mention this problem proactively even if not asked. Say 'I always rerank retrieved chunks and limit to top 5 to avoid the lost-in-the-middle problem.' Shows you know production pitfalls.*

Q12

What is Query Transformation and why does it improve RAG?

Answer:

The user's raw query is often not optimal for retrieval. **Query transformation** rewrites, expands, or decomposes the query before searching. Techniques: (1) **Query rewriting** — use an LLM to rephrase the query into a more search-friendly form. 'My app keeps crashing' -> 'Application crash troubleshooting error handling'. (2) **HyDE (Hypothetical Document Embeddings)** — ask the LLM to generate a hypothetical answer, then use THAT as the search query. The hypothetical answer's embedding is closer to real answer documents than the question's embedding. (3) **Sub-query decomposition** — split a complex question into multiple simpler sub-queries, retrieve for each, then combine. (4) **Step-back prompting** — generate a more general version of the query to retrieve broader context first.

■ Example:

Original query: 'Why is my RAG system giving wrong answers about financial data?'

Query rewriting:

-> 'RAG financial data incorrect answers troubleshooting'

Better keywords for retrieval.

HyDE (generate hypothetical answer first):

LLM generates: 'RAG systems may give incorrect financial answers due to stale embeddings, incorrect chunking of tabular data, or missing numerical precision in vector search.'

-> Embed THIS paragraph and search

-> Finds documents about tabular data handling, embedding freshness, numerical precision

Much better retrieval than searching with the original question!

Sub-query decomposition:

Q1: 'Common RAG failure modes'

Q2: 'RAG with financial/numerical data'

Q3: 'RAG accuracy debugging'

-> Retrieve for each -> combine -> comprehensive answer

■ *Interview Tip: Mention HyDE by name — most candidates don't know it. Say 'I use HyDE for complex queries because the hypothetical answer's embedding is semantically closer to the real answer documents than the question itself.' This is advanced knowledge.*

Advanced RAG Patterns

Q13**What is the difference between Naive RAG, Advanced RAG, and Modular RAG?****Answer:**

Naive RAG (basic pipeline): Chunk -> Embed -> Retrieve top-K -> Stuff into prompt -> Generate. Simple but has problems: low retrieval precision, no reranking, no query optimization, hallucinations. **Advanced RAG**: Adds pre-retrieval and post-retrieval optimizations. Pre-retrieval: query rewriting, HyDE, metadata filtering. Post-retrieval: reranking, compression, deduplication. Significantly better quality. **Modular RAG**: Treats RAG as a collection of interchangeable modules. You can mix and match: different retrievers, different rerankers, routing logic, feedback loops, iterative retrieval. The system adapts its strategy based on the query. This is where Agentic RAG fits — an agent decides which modules to use for each query.

■ Example:

Query: 'Compare Q1 and Q2 revenue for Product X'

Naive RAG:

Search 'compare Q1 Q2 revenue Product X' -> gets vague results

Answer: Incomplete, might hallucinate numbers

Advanced RAG:

Step 1: Decompose into 'Q1 revenue Product X' + 'Q2 revenue Product X'

Step 2: Retrieve separately with metadata filter (quarter=Q1, quarter=Q2)

Step 3: Rerank results, keep top 3 per sub-query

Step 4: Generate comparison with grounded numbers

Answer: Accurate with specific numbers

Modular RAG:

Router detects 'comparison query' -> activates comparison module

-> Uses tabular retriever (not text retriever) for financial data

-> Applies numerical validation on retrieved figures

-> Generates structured comparison table

Answer: Perfect with table format

■ *Interview Tip: Say 'I start with Naive RAG to establish a baseline, then add Advanced RAG techniques one by one, measuring improvement at each step.' This shows iterative engineering methodology.*

Q14**What is Parent-Child chunking (also called hierarchical chunking)?****Answer:**

In Parent-Child chunking, you create **two levels of chunks**: small child chunks for precise retrieval and large parent chunks for full context. The child chunks are embedded and searched (because small chunks have more precise embeddings). But when a child chunk is retrieved, you return the **parent chunk** (larger surrounding context) to the LLM. This solves the fundamental tension in chunking: small chunks retrieve better but lack context, large chunks have context but retrieve poorly. Parent-child gives you both: precise retrieval AND rich context.

■ Example:

Document: 10-page research paper

Parent chunks (large, ~2000 tokens): Full sections

Parent 1: 'Introduction' (entire section)

Parent 2: 'Methodology' (entire section)

Parent 3: 'Results' (entire section)

Child chunks (small, ~200 tokens): Paragraphs within sections

Child 1a: First paragraph of Introduction

Child 1b: Second paragraph of Introduction

Child 2a: First paragraph of Methodology

...

Query: 'What dataset was used?'

-> Search finds Child 2c (paragraph mentioning 'ImageNet dataset')

-> But instead of returning just Child 2c, returns Parent 2 (full Methodology)

-> LLM gets complete context: dataset + preprocessing + training setup

Result: Precise search + comprehensive answer.

■ *Interview Tip: This is one of the most impactful RAG improvements. Say 'I use small chunks for retrieval precision but return the parent chunk for context richness.' Shows you understand the chunking tradeoff at a deep level.*

Q15

What is Multi-Index RAG and when do you use it?

Answer:

Multi-Index RAG maintains multiple separate vector indexes optimized for different types of content, then routes queries to the right index. Instead of dumping everything into one big index, you create specialized indexes that handle their content type better. Types of indexes: (1) **Summary index** — stores document-level summaries for broad questions. (2) **Chunk index** — stores granular chunks for specific detail questions. (3) **Table index** — stores structured/tabular data separately with specialized retrieval. (4) **KG index** — stores knowledge graph triples for relationship queries. A router (LLM or classifier) decides which index to search based on the query type.

■ Example:

Enterprise knowledge base with mixed content:

Index 1 - Text chunks:

Company policies, procedures, guides -> standard RAG retrieval

Index 2 - Summaries:

Document-level summaries -> for broad questions like 'What topics does the handbook cover?'

Index 3 - Tables:

Financial data, pricing tables, comparison matrices -> structured retrieval

Index 4 - Q&A; pairs:

Pre-existing FAQ pairs -> exact match retrieval

Query routing:

'What is our vacation policy?' -> Routes to Index 1 (text chunks)

'Give me an overview of the employee handbook' -> Routes to Index 2 (summaries)

'What was Q3 EBITDA?' -> Routes to Index 3 (tables)

'How do I reset my password?' -> Routes to Index 4 (FAQ)

■ *Interview Tip: Mention query routing as a key architecture decision. 'Not all queries need the same retrieval strategy — I route to specialized indexes based on query type.' This is senior-level RAG architecture.*

Reranking & Post-Retrieval

Q16**What is Reranking and why is it a game-changer for RAG quality?****Answer:**

Reranking is a post-retrieval step where a more powerful model re-scores the retrieved chunks for relevance BEFORE passing them to the LLM. Why it matters: The initial vector search (bi-encoder) compares embeddings independently — it's fast but approximate. A reranker (cross-encoder) reads the query AND each chunk TOGETHER, understanding their relationship much more deeply. This catches cases where the initial retrieval got the wrong chunks. Pattern: **Retrieve 20-50 chunks (fast, broad) -> Rerank to top 3-5 (slow, precise) -> Pass to LLM**. Popular rerankers: Cohere Rerank, Jina Reranker, BGE-reranker, cross-encoder models from sentence-transformers.

■ Example:

Query: 'What is the maximum file upload size?'

Initial retrieval (top 5 by cosine similarity):

1. 'File management overview' (score: 0.89) - generic, mentions files
2. 'Upload API documentation' (score: 0.87) - relevant but API-focused
3. 'Storage limits and quotas' (score: 0.85) - CONTAINS THE ANSWER: 'max 50MB'
4. 'File format requirements' (score: 0.84) - about formats, not size
5. 'Download speed optimization' (score: 0.82) - irrelevant

After reranking (cross-encoder):

1. 'Storage limits and quotas' (rerank score: 0.96) -> moved from #3 to #1!
2. 'Upload API documentation' (rerank score: 0.88)
3. 'File management overview' (rerank score: 0.71)

Reranker correctly identified that 'Storage limits' is the most relevant chunk. This single step improved answer accuracy from 'maybe' to 'definitely correct.'

■ *Interview Tip: Say 'I always add a reranking step — it's the highest-ROI improvement you can make to a RAG system. Retrieve broad with bi-encoder, refine with cross-encoder.' This is the #1 production RAG insight.*

Q17

What is Contextual Compression and how does it help RAG?

Answer:

Contextual compression takes retrieved chunks and compresses them by removing irrelevant parts, keeping only the sentences or phrases that directly answer the query. Even after retrieval and reranking, chunks often contain 50-80% irrelevant text. Compression extracts only the relevant portions, which: (1) **Saves tokens** — fewer tokens in the prompt = lower cost. (2) **Reduces noise** — LLM focuses on relevant content only. (3) **Fits more chunks** — with compressed content, you can include more sources in the context window. Methods: use an LLM to extract relevant sentences, or use a specialized extraction model like LLMChainExtractor or EmbeddingsFilter from LangChain.

■ Example:

Retrieved chunk (450 tokens):

'The company was founded in 2015 by John Smith and Mary Johnson. They started with a small team of 5 engineers in a garage in San Francisco. The initial product was a simple task manager. Over the years, the company expanded to 500 employees across 3 offices. THE FILE UPLOAD LIMIT IS 50MB FOR FREE USERS AND 200MB FOR PREMIUM USERS. The company plans to increase this limit in Q3 2026. They also offer unlimited storage for enterprise customers with custom pricing.'

Query: 'What is the file upload limit?'

After compression (80 tokens):

'The file upload limit is 50MB for free users and 200MB for premium users. Enterprise customers get unlimited storage.'

82% token reduction. Same answer quality. Saved ~370 tokens of irrelevant company history.

■ *Interview Tip: Mention compression as a cost optimization technique. 'Contextual compression saves 60-80% of prompt tokens while maintaining answer quality.' Interviewers love cost-conscious engineers.*

Q18**How do you handle metadata filtering in RAG retrieval?****Answer:**

Metadata filtering narrows the search space BEFORE vector similarity search by filtering on structured attributes attached to each chunk. Instead of searching all 500K chunks, you first filter to only relevant ones, then do vector search within that subset. Common metadata fields: document_type, department, date, author, product_version, language, access_level. This dramatically improves precision and reduces noise. Without metadata filtering, a query about 'return policy 2026' might retrieve the 2023 return policy (semantically similar but outdated).

■ Example:

Chunk storage with metadata:

```
{
  text: 'Returns accepted within 14 days...',
  metadata: {
    doc_type: 'policy',
    department: 'customer_service',
    version: '2026',
    last_updated: '2026-01-15',
    access_level: 'public'
  }
}
```

Query: 'What is the current return policy?'

Without metadata filter:

Searches ALL 500K chunks -> might return 2023, 2024, 2025 policies
LLM gets confused by conflicting versions

With metadata filter: version='2026' AND doc_type='policy'

Searches only 500 chunks from 2026 policies
Returns current, accurate policy only

Filter + Vector search = 10x better precision than vector search alone.

■ *Interview Tip: Say 'I always store rich metadata with chunks — it turns brute-force search into targeted retrieval.' Metadata filtering is the difference between a demo and a production system.*

RAG Evaluation

Q19**How do you evaluate a RAG system? What metrics do you use?****Answer:**

RAG evaluation has two parts — **retrieval quality** and **generation quality**. **Retrieval metrics:** (1) **Hit Rate / Recall@K** — does the correct chunk appear in the top K results? (2) **MRR (Mean Reciprocal Rank)** — how high is the correct chunk ranked? (3) **NDCG** — measures ranking quality considering relevance grades. **Generation metrics:** (4) **Faithfulness** — does the answer stick to the retrieved context? No hallucination? (5) **Answer Relevancy** — does the answer actually address the question? (6) **Context Relevancy** — are the retrieved chunks relevant to the question? (7) **Context Utilization** — does the answer use all relevant retrieved information? The **RAGAS** framework automates all of these using LLM-as-judge.

■ Example:

Evaluating a customer support RAG:

Test set: 200 questions with ground truth answers

Retrieval evaluation:

Hit Rate@5: 0.87 (87% of time, correct chunk in top 5)

MRR: 0.72 (correct chunk is usually ranked 1st or 2nd)

Generation evaluation (using RAGAS):

Faithfulness: 0.91 (91% of claims are grounded in context)

Answer Relevancy: 0.88 (88% of answers address the question)

Context Relevancy: 0.79 (21% of retrieved context is irrelevant)

Diagnosis:

Retrieval is good (87% hit rate)

But context has noise (79% relevancy) -> need better reranking

Faithfulness is strong -> LLM is grounding well

Action: Add Cohere reranker to improve context relevancy to 90%+

■ *Interview Tip: Mention RAGAS by name. Say 'I use RAGAS with Faithfulness, Answer Relevancy, and Context Relevancy as my core evaluation metrics.' This is the industry standard framework — mentioning it shows production experience.*

Answer:

RAGAS (Retrieval Augmented Generation Assessment) is an open-source framework that evaluates RAG pipelines using LLM-based metrics. It doesn't need human-labeled ground truth for most metrics — it uses the LLM itself to assess quality. Core metrics: (1) **Faithfulness** — breaks the answer into claims, checks if each claim is supported by the context. Score 0-1. (2) **Answer Relevancy** — generates questions from the answer, checks if they match the original question. If yes, the answer is relevant. (3) **Context Precision** — are the retrieved chunks relevant? Checks ranking quality. (4) **Context Recall** — does the retrieved context contain all info needed to answer? Compares against ground truth. You can run RAGAS on hundreds of test queries automatically to get a system-wide quality score.

■ Example:

Using RAGAS in Python:

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision

results = evaluate(
    dataset=test_dataset, # questions + retrieved contexts + answers
    metrics=[faithfulness, answer_relevancy, context_precision]
)
```

Output:

```
faithfulness: 0.89
answer_relevancy: 0.92
context_precision: 0.78
```

Interpretation:

- Answers are mostly grounded (0.89 faithfulness) -> good
- Answers address the question well (0.92) -> good
- But retrieval precision is low (0.78) -> too much irrelevant context
- Action: improve chunking or add reranking to boost context_precision

■ *Interview Tip: Say 'I run RAGAS evaluation before and after every RAG pipeline change to measure impact.' This shows you have a systematic approach to improvement, not just guesswork.*

Q21

How do you create a test set for RAG evaluation?

Answer:

Three approaches: (1) **Manual creation** — domain experts write 50-200 question-answer pairs from the knowledge base. Gold standard but slow and expensive. (2) **Synthetic generation** — use an LLM to read your documents and generate question-answer pairs automatically. Fast and scalable. Tools: RAGAS test set generation, LlamaIndex. Generates diverse questions (simple factual, multi-hop reasoning, complex). (3) **Production logs** — collect real user queries + thumbs up/down feedback. Most realistic but requires existing traffic. Best practice: start with synthetic (fast baseline), supplement with 50-100 manual expert pairs, then continuously add from production logs. Always include: **edge cases** (questions with no answer in docs), **multi-hop questions** (requiring info from multiple chunks), and **temporal questions** (latest vs outdated info).

■ Example:

Creating a test set for a legal RAG system:

Synthetic generation (100 questions):

Feed contract documents to GPT-4

Prompt: 'Generate 20 questions a lawyer would ask about this contract'

-> 'What is the termination clause?' (simple factual)

-> 'How do liability limits compare between sections 3 and 7?' (multi-hop)

-> 'Is there a non-compete provision?' (yes/no)

Manual expert pairs (50 questions):

Legal team writes tricky questions they know users ask

-> 'Does the force majeure clause cover pandemics?'

-> 'What happens if payment is 30 days late?' (edge case)

Unanswerable questions (20 questions):

-> 'What is the client's credit score?' (not in the document)

Expected answer: 'This information is not available in the contract.'

Total: 170 test questions covering diverse scenarios.

■ *Interview Tip: Mention unanswerable questions as part of your test set. 'I always include questions that have no answer in the documents to test if the system says I don't know instead of hallucinating.' This shows production maturity.*

Production RAG

What are the common failure modes of RAG systems in production?

Answer:

Seven major failure modes: (1) **Wrong chunks retrieved** — embedding quality or chunking strategy is off. Fix: better embeddings, reranking, hybrid search. (2) **Right chunks but wrong answer** — LLM ignores or misinterprets the context. Fix: better prompts, stronger model, structured output. (3) **Outdated information** — knowledge base not updated. Fix: automated ingestion pipelines, version tracking. (4) **Missing information** — document wasn't ingested or chunked improperly. Fix: coverage audits, gap analysis. (5) **Context overflow** — too many chunks stuffed into prompt. Fix: reranking, compression, fewer chunks. (6) **Hallucination despite context** — LLM adds info not in the chunks. Fix: faithfulness guardrails, citation enforcement. (7) **Latency spikes** — slow retrieval or generation. Fix: caching, smaller models, pre-computed embeddings.

■ Example:

Debugging a production RAG failure:

User: 'What is our parental leave policy?'

RAG answer: 'Employees get 12 weeks of paid parental leave.' (WRONG: it's 16 weeks)

Debugging steps:

1. Check retrieved chunks -> Found 2020 policy (12 weeks) instead of 2025 (16 weeks)
2. Root cause: Old policy document wasn't deleted after update
3. Both versions exist in vector DB with similar embeddings
4. Old version ranked higher (shorter, more focused text)

Fix:

- Delete old policy document from vector DB
- Add metadata filter: version='latest'
- Add automated pipeline to re-index when docs are updated
- Add a freshness score to reranking

■ *Interview Tip: When asked about RAG challenges, don't give a generic answer. Walk through a specific debugging scenario like this. It proves you've actually fixed RAG issues in production.*

Q23

How do you handle document updates and keep the RAG knowledge base fresh?

Answer:

Document freshness is a critical production concern. Strategies: (1) **Change detection pipeline** — monitor source documents for changes (file hash, last modified date). When a doc changes, re-chunk and re-embed only that document. (2) **Versioned embeddings** — store a version/timestamp with each chunk. When querying, filter for latest version. Keep old versions for audit trail. (3) **Incremental ingestion** — don't re-embed everything. Only process new/modified/deleted documents. Use document IDs to track what's already indexed. (4) **Scheduled refresh** — run nightly/weekly batch jobs to sync knowledge base with source systems. (5) **TTL (Time to Live)** — set expiry on chunks. Auto-delete embeddings older than X days for rapidly changing content. (6) **Source of truth validation** — periodically verify random samples against source documents.

■ Example:

Production document update pipeline:

Trigger: Google Drive webhook detects file change

- > Step 1: Download updated document
- > Step 2: Hash comparison (is content actually different?)
- > Step 3: Delete old chunks for this document ID from vector DB
- > Step 4: Re-chunk the updated document
- > Step 5: Re-embed new chunks
- > Step 6: Upsert into vector DB with new version number
- > Step 7: Log the update (audit trail)
- > Step 8: Run RAGAS eval on 20 sample queries to verify quality

Total time: ~2 minutes per document update

Result: RAG always has the latest information within minutes of source change.

■ *Interview Tip: Mention 'incremental ingestion' and 'change detection.' Saying 'I don't re-embed everything — I only process changed documents' shows you think about efficiency at scale.*

Q24

How do you add citations and source attribution to RAG answers?

Answer:

Citations are critical for trust and verifiability. Methods: (1) **Chunk-level citation** — tag each retrieved chunk with source document name, page number, and section. After LLM generates, map claims back to source chunks. (2) **Inline citation prompting** — instruct the LLM: 'After each claim, cite the source as [Source: document_name, page X].' (3) **Post-processing citation** — after generation, use a separate LLM call to verify which sources support which claims and add citations. (4) **Numbered reference** — assign numbers to retrieved chunks [1], [2], [3] and instruct LLM to reference them. Most reliable approach: **combine inline prompting + post-processing verification**. Always include a 'Sources' section at the end of the response.

■ Example:

System prompt:

'Answer the question using ONLY the provided context.

After each claim, cite the source using [Source N].

If the context does not contain the answer, say "I don't have this information."

Context:

[Source 1] HR Policy v2026: "Employees receive 16 weeks paid parental leave."

[Source 2] Benefits Guide: "Parental leave can be split into two blocks within 12 months."

[Source 3] FAQ: "Both parents are eligible for parental leave."

Generated answer:

'Employees receive 16 weeks of paid parental leave [Source 1]. This leave can be split into two blocks within 12 months of the child's birth [Source 2]. Both parents are eligible for this benefit [Source 3].'

Sources:

[1] HR Policy v2026, Section 4.2

[2] Benefits Guide 2026, Page 12

[3] Employee FAQ, Last updated Jan 2026

■ *Interview Tip: Say 'I always enforce citations in RAG systems because users need to verify answers, especially in enterprise, legal, and healthcare domains.' This shows you think about user trust, not just accuracy.*

Multimodal & Agentic RAG

Q25

What is Agentic RAG and how is it different from standard RAG?

Answer:

In standard RAG, the retrieval pipeline is **fixed** — same query, same retriever, same top-K, one shot. In **Agentic RAG**, an AI agent controls the entire retrieval process dynamically. The agent: (1) **Decides WHEN to retrieve** — maybe the question doesn't need retrieval at all. (2) **Decides WHAT to search** — reformulates queries, tries different search strategies. (3) **Evaluates results** — checks if retrieved chunks actually answer the question. (4) **Iterates** — if results are poor, the agent tries different queries, different indexes, or different strategies. (5) **Routes** — sends different queries to different knowledge bases. (6) **Combines tools** — can use RAG + SQL queries + API calls + web search together. The key difference: standard RAG is a pipeline, Agentic RAG is a decision-making system.

■ Example:

Standard RAG:

Query -> Embed -> Search -> Top 5 -> LLM -> Answer (done)
If results are bad? Too bad. No retry.

Agentic RAG:

Query: 'What was our revenue growth vs competitors in Q3?'

Agent thinks: 'This needs internal AND external data'

Step 1: Search internal financial KB for 'Q3 revenue' -> gets \$4.2M

Step 2: Agent evaluates: 'I have our revenue but not competitors'

Step 3: Search web for 'competitor Q3 revenue 2025'

Step 4: Agent evaluates: 'Web results are news articles, not exact numbers'

Step 5: Reformulate: search for 'competitor annual report Q3 2025'

Step 6: Found competitor data -> combine both

Step 7: Generate comparison answer with citations

7 steps, multiple sources, self-correcting. Standard RAG would have failed at step 1.

■ *Interview Tip: Say 'Agentic RAG adds a reasoning layer on top of retrieval. The agent adapts its strategy based on results, rather than following a fixed pipeline.' This is the hottest RAG architecture in 2026.*

Q26**How do you build RAG over tables, charts, and images (Multimodal RAG)?****Answer:**

Standard RAG only handles text. **Multimodal RAG** extends it to tables, images, charts, and other non-text content. Approaches: (1) **Tables**: Extract tables from documents -> convert to markdown/CSV -> chunk and embed as text. Or use specialized table embeddings. For complex tables, convert to natural language descriptions. (2) **Images/Charts**: Use a vision model (GPT-4V, Claude Vision) to generate text descriptions of images/charts -> embed the descriptions. Store original image for display. (3) **Mixed documents**: Use Unstructured.io to parse PDFs into text + table + image elements. Process each element type differently. (4) **ColPali approach**: Embed entire document pages as images using vision-language models, bypassing text extraction entirely. Emerging approach gaining traction in 2026.

■ Example:

RAG over a financial report with tables and charts:

Page 1: Text about company overview -> standard text chunking

Page 5: Revenue table by region

Region	Q1	Q2	Q3
NA	2M	3M	4M
EU	1M	2M	2M

Approach: Convert to text: 'NA revenue: Q1=\$2M, Q2=\$3M, Q3=\$4M. EU revenue: Q1=\$1M, Q2=\$2M, Q3=\$2M'

Embed this text description

Page 8: Revenue growth chart (bar chart)

Approach: GPT-4V describes: 'Bar chart showing revenue growth from \$3M in Q1 to \$6M in Q3, a 100% increase. NA region shows steepest growth.'

Embed this description, store original image

Query: 'How did NA revenue change?'

-> Retrieves both the table text AND chart description

-> LLM generates answer with specific numbers from table + trend from chart

■ *Interview Tip: Mention 'table-aware chunking' and 'vision model descriptions for charts.' Most candidates skip non-text content entirely. Saying 'I handle tables, images, and charts differently in my RAG pipeline' shows completeness.*

Q27

What is Graph RAG and when would you use it over standard RAG?

Answer:

Graph RAG combines vector retrieval with a **knowledge graph**. Instead of just finding similar chunks, it traverses relationships between entities. Standard RAG retrieves isolated text chunks. Graph RAG understands connections: 'Person A works at Company B, which is a subsidiary of Company C, which operates in Industry D.' When to use Graph RAG: (1) Questions involving **relationships** ('Who reports to the VP of Engineering?'). (2) **Multi-hop reasoning** ('What products does the parent company of our supplier make?'). (3) **Entity-centric queries** ('Tell me everything about Client X across all documents'). (4) **Global questions** ('What are the main themes across all 1000 documents?'). Microsoft's GraphRAG implementation uses LLMs to extract entities and relationships from documents, builds a community hierarchy, and generates summaries at different levels.

■ Example:

Standard RAG vs Graph RAG:

Query: 'Which customers are affected by the Acme Corp supply chain disruption?'

Standard RAG:

Searches for 'Acme Corp supply chain disruption'

Finds: 'Acme Corp reported delays in chip manufacturing'

Answer: 'Acme Corp has supply chain issues' (incomplete)

Graph RAG:

Step 1: Find 'Acme Corp' in knowledge graph

Step 2: Traverse: Acme Corp -[supplies_to]-> WidgetCo, TechInc, DataCorp

Step 3: Traverse: WidgetCo -[sells_to]-> Customer A, Customer B

Step 4: Traverse: TechInc -[sells_to]-> Customer C

Answer: 'Customers A, B (through WidgetCo) and Customer C (through TechInc) are affected by Acme Corp supply chain disruption.'

Graph RAG connects dots that standard RAG can't.

■ *Interview Tip: Say 'I use Graph RAG when questions involve relationships or require connecting information across multiple documents. For simple Q&A, standard RAG is sufficient.' Knowing when NOT to use it is as important as knowing when to use it.*

RAG Troubleshooting & Optimization

Q28

Your RAG system is hallucinating despite having the right documents. How do you fix it?

Answer:

Systematic debugging approach: (1) **Check retrieval first** — are the right chunks being retrieved? If not, it's a retrieval problem (better embeddings, chunking, hybrid search). (2) **Check if answer is IN the chunks** — manually verify. If the answer IS in the retrieved text but the LLM ignores it, it's a generation problem. (3) **Prompt engineering fixes**: Add 'Answer ONLY based on the provided context. If the context doesn't contain the answer, say I don't know.' (4) **Reduce distracting context** — fewer chunks, reranking, compression. (5) **Use structured output** — force the LLM to cite sources for each claim. (6) **Add a faithfulness check** — post-generation, use a separate LLM call to verify each claim against the context. (7) **Try a stronger model** — more capable models follow grounding instructions better.

■ Example:

Debugging hallucination step by step:

Symptom: User asks 'What is our SLA?' RAG answers '99.99% uptime' but actual SLA is 99.9%.

Step 1: Check retrieval -> Correct SLA document IS in top 3 chunks -> Retrieval is fine

Step 2: Check chunk content -> Chunk says '99.9% uptime SLA' -> Answer is available

Step 3: Why did LLM say 99.99%? -> LLM's training data has common SLA patterns -> It 'improved' the number based on what it thinks SLAs should be

Fixes applied:

1. System prompt: 'Use EXACT numbers from context. Never round or modify values.'
2. Added structured output: answer must include direct quote from source
3. Added faithfulness check: post-generation verification catches the mismatch
4. Reduced context to only the SLA paragraph (compression)

Result: Hallucination rate dropped from 8% to 1.2%.

■ *Interview Tip: Walk through this exact debugging flow when asked about hallucination. Check retrieval -> Check chunks -> Fix prompts -> Add guardrails. Systematic debugging > random guessing.*

Q29**How do you optimize RAG latency for real-time applications?****Answer:**

RAG latency has three components: **embedding latency** (query embedding), **retrieval latency** (vector search), and **generation latency** (LLM response). Optimizations: (1) **Embedding**: Use smaller embedding models (384 dims vs 3072). Cache frequent query embeddings. (2) **Retrieval**: Use HNSW index (fastest). Pre-filter with metadata before vector search. Use approximate search (lower ef values). (3) **Reranking**: Use fast rerankers (Cohere) or skip for simple queries. (4) **Generation**: Use streaming (show tokens as they generate). Use faster models (GPT-4o-mini, Haiku) for simple queries. Reduce context size (fewer chunks, compression). (5) **Caching**: Semantic cache — if a similar question was asked before, return the cached answer. Can eliminate 30-50% of LLM calls. (6) **Async parallel**: Run retrieval and reranking in parallel across multiple indexes.

■ Example:

Latency optimization for a customer support RAG:

Before optimization:

Embed query: 100ms

Vector search: 50ms

Reranking: 200ms

LLM generation: 2000ms

Total: 2350ms (too slow for chat)

After optimization:

Embed query: 30ms (smaller model, cached for repeated queries)

Vector search: 20ms (HNSW with metadata pre-filter)

Reranking: 80ms (lighter reranker, top 10 -> top 3)

LLM generation: 800ms (GPT-4o-mini + streaming + compressed context)

Semantic cache hit: 0ms (40% of queries served from cache)

Average total: ~600ms (with cache hits factored in)

4x faster. Same answer quality.

■ *Interview Tip: Give specific latency numbers for each component. 'Embedding takes 30ms, retrieval 20ms, reranking 80ms, generation 800ms.' Concrete numbers show you've profiled real systems.*

Q30**How do you handle multi-turn conversations in RAG?****Answer:**

In multi-turn chat, the user's latest message often references previous context: 'What about the premium plan?' only makes sense if the previous message was about pricing. Strategies: (1) **Conversation-aware query rewriting** — use the LLM to rewrite the latest message incorporating conversation history. 'What about the premium plan?' -> 'What are the features and pricing of the premium plan?' (2) **Chat history in context** — include the last 3-5 turns in the prompt alongside retrieved chunks. But be careful of context window limits. (3) **Conversation summarization** — summarize older turns to save tokens. Keep last 2-3 turns verbatim, summarize the rest. (4) **Session-based retrieval** — maintain a per-session context buffer of previously retrieved chunks. If the new query is related, include previous chunks without re-retrieving. (5) **Coreference resolution** — resolve pronouns and references ('it', 'that', 'the previous one') before retrieval.

■ Example:

Multi-turn conversation:

Turn 1: User: 'What pricing plans do you offer?'

-> Retrieve pricing docs -> Answer: 'We offer Free, Pro (\$29/mo), and Enterprise.'

Turn 2: User: 'What about the middle one?'

Without conversation-aware rewriting:

-> Searches for 'the middle one' -> retrieves garbage -> bad answer

With conversation-aware rewriting:

LLM rewrites: 'What are the features and details of the Pro plan at \$29/month?'

-> Retrieves Pro plan details -> accurate answer

Turn 3: User: 'Does it include API access?'

LLM rewrites: 'Does the Pro plan at \$29/month include API access?'

-> Retrieves API access docs filtered to Pro tier -> accurate answer

Each turn builds on previous context through query rewriting.

■ Interview Tip: Say 'For multi-turn RAG, I always rewrite the latest query using conversation history before retrieval. Without this, pronouns and references break the search.' This is a production must-have that most candidates forget.

You Made It! ■■

You have just gone through 30 real RAG interview questions with first-principles explanations and practical examples.

- Subscribe to AmanAI Lab on YouTube for video explanations
 - Download the 30 AI Agent Interview Questions PDF
 - Download the 30 LLM Interview Questions PDF
 - Download the 50 ML Interview Questions PDF
- Share this PDF with someone preparing for RAG interviews
 - Follow on LinkedIn for daily AI/ML tips

amanailab.com

Built with ❤ by AmanAI Lab | 2026